

EagleEye: Intelligent Edge AI Surveillance

Daily Change Report and Development History

Project repository: `fyp-eagle-eye`

Report generated: 2 May 2026

Sources: `git log` (full history) and working-tree diff for 2 May 2026

Abstract

This document records project evolution in a professional, date-ordered format. Historical entries are derived from committed changes in the version-control system. The entry for **2 May 2026** additionally summarizes enhancements present in the working tree at report generation time (including changes not yet committed), covering firmware live-preview transport, mobile application updates, and gallery administration features.

Contents

1	Purpose and conventions	2
2	Chronological development summary	2
2.1	25 October 2025	2
2.2	21 November 2025	2
2.3	26 November 2025	2
2.4	8 December 2025	2
2.5	10 December 2025	2
2.6	14 December 2025	2
2.7	10 February 2026	2
2.8	14 February 2026	2
2.9	16 February 2026	3
2.10	17 February 2026	3
2.11	18 February 2026	3
2.12	6 March 2026	3
2.13	21 April 2026	3
2.14	2 May 2026 (committed)	3
2.15	2 May 2026 (working tree — detailed technical delta)	3
3	Suggested maintenance	4
4	Compilation	4

1 Purpose and conventions

- **Scope:** EagleEye spans ESP32-CAM firmware, a Python MQTT–cloud bridge, a React Native (Expo) mobile client, model training assets, and documentation.
- **Sources:** Commit subjects and bodies are paraphrased for clarity; hashes identify the corresponding revision where applicable.
- **Updates:** Append new dated subsections when material changes land in `main`. Recompile this `.tex` file to refresh the PDF.

2 Chronological development summary

2.1 25 October 2025

Milestone: repository bootstrap. Initial README and project upload; early ESP32-CAM bring-up on university Wi-Fi infrastructure.

2.2 21 November 2025

Milestone: on-device inference. TensorFlow Lite pipeline validated; interactive verification via DumbDisplay (tap-to-classify prototype).

2.3 26 November 2025

Maintenance. Minor commit (no extended message in history).

2.4 8 December 2025

End-to-end IoT path. Human detection with approximately 712 ms latency; intruder JPEG relayed via MQTT; Cloudinary storage with metadata URLs persisted in Firebase Realtime Database. Modular layout: camera sketch with `EagleEye_IoT.h` for connectivity.

Security hygiene. `.gitignore` extended to exclude secrets (`.env`, `secrets.h`, `serviceAccountKey.json`).

2.5 10 December 2025

Documentation. README expanded to describe system architecture and usage.

2.6 14 December 2025

Local MQTT and payload efficiency. Mosquitto-based local broker; bridge subscribed for diagnostics; ESP32 transmits raw binary JPEG (larger MQTT buffer, tuned thresholds and cooldown). Supporting assets: Mosquitto configuration, training notebooks, quality simulation utilities, dataset maintenance.

2.7 10 February 2026

Maintenance. Minor commit.

2.8 14 February 2026

Custom tiny model family. In-house training artifacts under a dedicated model directory; reference to a ~ 106 ms variant and Arduino export layout.

2.9 16 February 2026

Greyscale TinyML deployment (46847c76). INT8 CNN at $48 \times 48 \times 1$; RGB565 capture with software greyscale for inference; color evidence uploads unchanged; tensor arena reduced to 100 kB; simplified capture path in `EagleEye_IoT.h`.

2.10 17 February 2026

Command and control (f803947c). Mobile: tabbed navigation, dashboard with arm/disarm and latest alert preview, heartbeat-driven online/offline badge. Bridge: Firebase listener on `config/armed`, intrusion gating when disarmed, periodic `status/heartbeat` updates. Manual start guide updated for USB/ADB workflow.

2.11 18 February 2026

Imaging pipeline (c79c10f7, 719dae9b). QVGA capture with center 240×240 crop prior to model resize—undistorted “digital zoom” for improved range; serial diagnostics updated.

2.12 6 March 2026

Academic milestone. Mid FYP presentation completed (status commit).

2.13 21 April 2026

Repository restructure (179ef260). Consolidated layout: `firmware/eagleeye-main`, `backend`, `mobile-app`, `model-training`, `datasets`, `docs`, `_archive`; `sketch` renamed to `eagleeye-main.ino`; manuals and README paths corrected.

Power-aware sensing (d87859da). PIR on GPIO 14, deep sleep, timed AI windows, armed-state respect, camera/Wi-Fi de-init before sleep; SD disabled to free the PIR pin.

2.14 2 May 2026 (committed)

Repository sync (7dc9cc93). Latest commit on record for this date (short message in history).

2.15 2 May 2026 (working tree — detailed technical delta)

The following items reflect substantive edits present locally at report time; integrate them into the next commit to keep history aligned.

1. **Firmware — HTTP snapshot API.** File `camera_web_server.h`: shared helper `eagleeye_grab_jpeg()` for single-frame JPEG extraction; `GET /capture` for mobile HTTP fallback; MJPEG stream on `GET /` unchanged. Include-guard typo corrected (erroneous leading character that broke preprocessing).
2. **Firmware — WebSocket preview.** New `eagleeye_ws.h`: binary JPEG push over WebSockets on port 81 (Arduino library: WebSockets by Markus Sattler). Reduces per-frame HTTP overhead versus repeated `/capture` polling.
3. **Firmware — control flow.** `eagleeye-main.ino`: call `eagleeye_ws_begin()` after HTTP server start; service `eagleeye_ws_loop()` each iteration; while WebSocket clients are connected, AI inference is deferred (camera bandwidth). **Latch reset:** when WebSocket clients drop from one to zero, or when browser MJPEG streaming ends, `image_sent_this_event`, `human_confirmed`, and `clear_scene_count` reset so detections resume after closing Live Monitor (previously the latch could remain set because the AI loop did not run during preview).

4. **Mobile — Live Monitor.** `LiveMonitorScreen.js`: primary path `ws://<camera-ip>:81` with binary frames rendered via `expo-image`; cleartext HTTP retained for camera URLs in `app.json` (`usesCleartextTraffic`). Legacy `react-native-webview` dependency removed from `package.json`.
5. **Mobile — Gallery administration.** `useAlerts.js`: `deleteAllAlerts()` loads full `alerts` node, enqueues `public_id` entries under `deletion_requests` for Cloudinary cleanup by `bridge.py`, then removes `alerts`. `GalleryScreen.js`: confirmed **Delete all** control with loading state and modal dismissal.

Note: Runtime JPEGs under `backend/captures/` may appear as untracked files during testing; exclude or gitignore per team policy. Local `secrets.h` changes (e.g. MQTT broker IP for hotspot vs. LAN) are not versioned by design.

3 Suggested maintenance

- After each merged feature, append a dated subsection (one paragraph + bullet list if needed) and re-run `pdflatex`.
- Keep commit messages descriptive; this report is easier to maintain when subjects follow `type(scope): summary` convention.

4 Compilation

```
cd docs/daily_changelog
pdflatex EagleEye_Daily_Change_Report.tex
pdflatex EagleEye_Daily_Change_Report.tex
```